



Sistemas Distribuidos I (75.74)

Mensajes, Grupos y Middlewares

Mensajes. Grupos de Comunicación. Middlewares. MOMs.

Docentes

- Pablo D. Roca
- Guido Albarello

- Gabriel Robles
- Franco Barreneche

- Tomás Nocetti
- Nicolás Zulaica

Agenda



- **Mensajes**
- Grupos
- Middlewares
- MOMs



Mensajes | Formato de Paquetes | Binario

- **Alta performance**
 - Tamaño de mensajes eficientes
 - Compresión puede no ser necesaria
- **Serialización**
 - Autogeneración de código
(e.g. Google Protobuf, Thrift, ASN.1)
 - No siempre existe soporte en todos los lenguajes
- **Interacción**
 - Cliente específico para cada aplicación
 - Decoder para interpretar los mensajes



Mensajes | Formato de Paquetes | Ejemplo (Protobuf)

person.proto

```
message Person {  
    required string name = 1;  
    required int32 id = 2;  
    optional string email = 3;  
}
```

Generación de Código

```
protoc -I=$SRC_DIR --cpp_out=$DST_DIR  
$SRC_DIR/person.proto
```

Serialización (Java)

```
Person john = Person.newBuilder()  
    .setId(1234)  
    .setName("John Doe")  
    .setEmail("jdoe@example.com")  
    .build();  
output = new  
FileOutputStream(args[0]);  
john.writeTo(output);
```

Deserialización (C++)

```
Person john;  
fstream input(argv[1],  
    ios::in | ios::binary);  
john.ParseFromIstream(&input);  
id = john.id();  
name = john.name();  
email = john.email();
```



Mensajes | Formato de Paquetes | Texto plano

- **Baja performance**
 - *Throughput* bajo
 - Compresión agrega *overhead*
- **Serialización**
 - Formatos *human-readable* (JSON, XML)
 - Serialización básica (e.g. HTTP, SMTP)
- **Interacción**
 - Cliente único si se conoce el protocolo (e.g. cURL + REST API)
 - Fácil de *debuggear*



Mensajes | Formato de Paquetes | Ejemplo (CURL)

```
user@hostname:~$ curl -H "Content-Type: application/json" \  
-X POST \  
-d '{"username":"lalala","password":"supersecurepass"}' \  
http://localhost:8080/api/login/
```

Request

```
POST /login HTTP/1.1  
Host: localhost:8080  
User-Agent: curl/7.58.0  
Accept: */*  
Content-Type: application/json  
Content-Length: 50  
  
{"username":"lalala","password":"supersecurepass"}
```



Mensajes | Longitud de Paquete

- **Bloques fijos**

- Cada dato a enviar posee una longitud fija
- Fácil de serializar
- Subóptimo con tipos de longitud variable (e.g. *strings*)

- **Bloques dinámicos**

- Separador para delimitar comienzo y terminación de un campo
- Longitud del tipo para delimitar longitud del campo
- Pequeño overhead al agregar *bytes* extra

- **Esquema mixto**

- Parámetros fijos (e.g. *integers*) no llevan delimitadores / longitud
- Parámetros variables llevan delimitadores / longitud



Mensajes | Longitud de Paquete (Ejemplo: TLV)

- Todos los parámetros siguen el formato Type - Length - Value
 - **Type:** Indica el tipo de dato / entidad. Tamaño fijo
 - **Length:** Longitud del value sin contar el tipo y el length. Tamaño fijo
 - **Value:** Dato a enviar. Tamaño variable. **Admite subtipos**
- Ejemplo: Encodear una dirección: Alsina 2B

Mini protocolo

```
Tipo 01: Integer (4 bytes)
Tipo 02: string (variable)
Tipo 03: Dirección (compuesto)
Tipo 03.01: Calle (string)
Tipo 03.02: Número (integer)
Tipo 03.03: Unidad (string)
```

Serialización

```
03 11 01 06 41 6c 73 69 6e 61 02 04 00 00
00 02 03 01 42
```

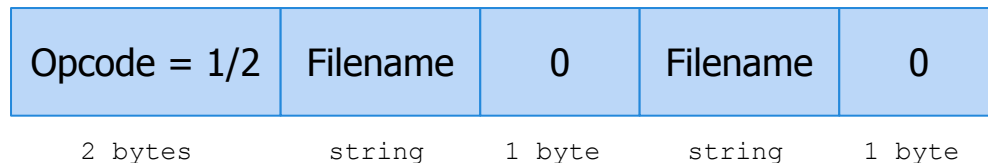
Serialización (desglose)

```
03 11 (Dirección)
    01 06 41 6c 73 69 6e 61 (Calle)
    02 04 00 00 00 02 (Número)
    03 01 42 (Unidad)
```




Mensajes | Longitud de Paquete (Ejemplo: TFTP)

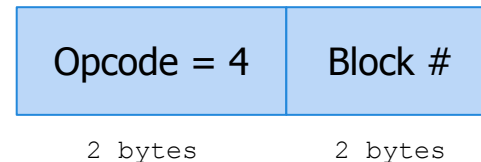
Read/Write Packet



Data Packet

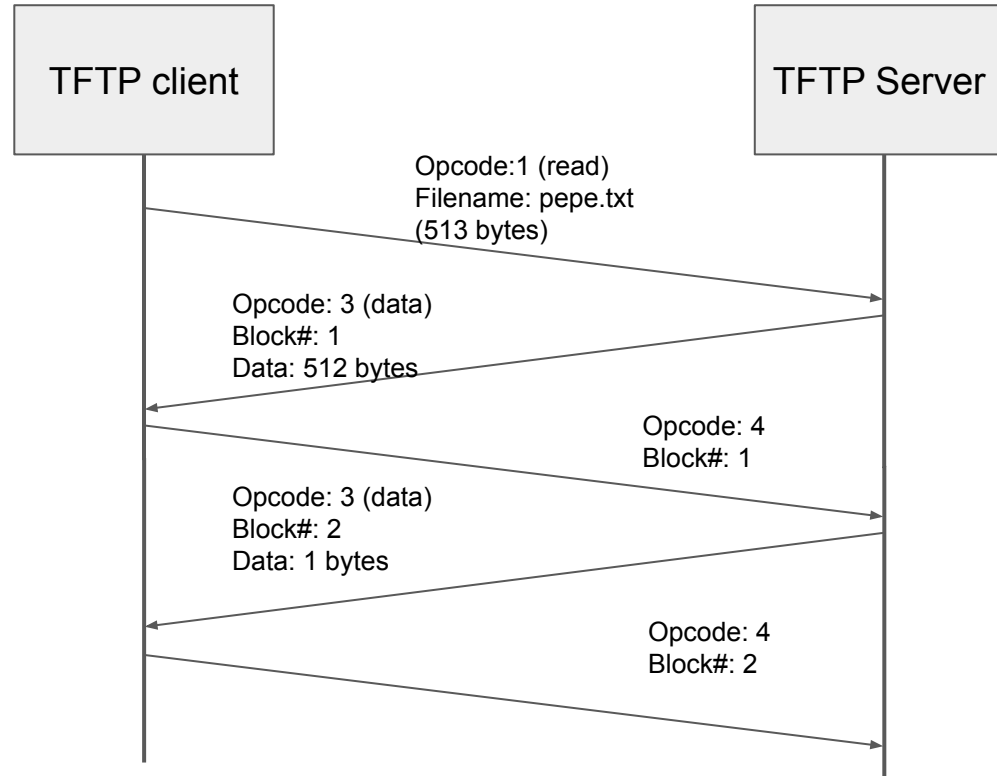


ACK Packet



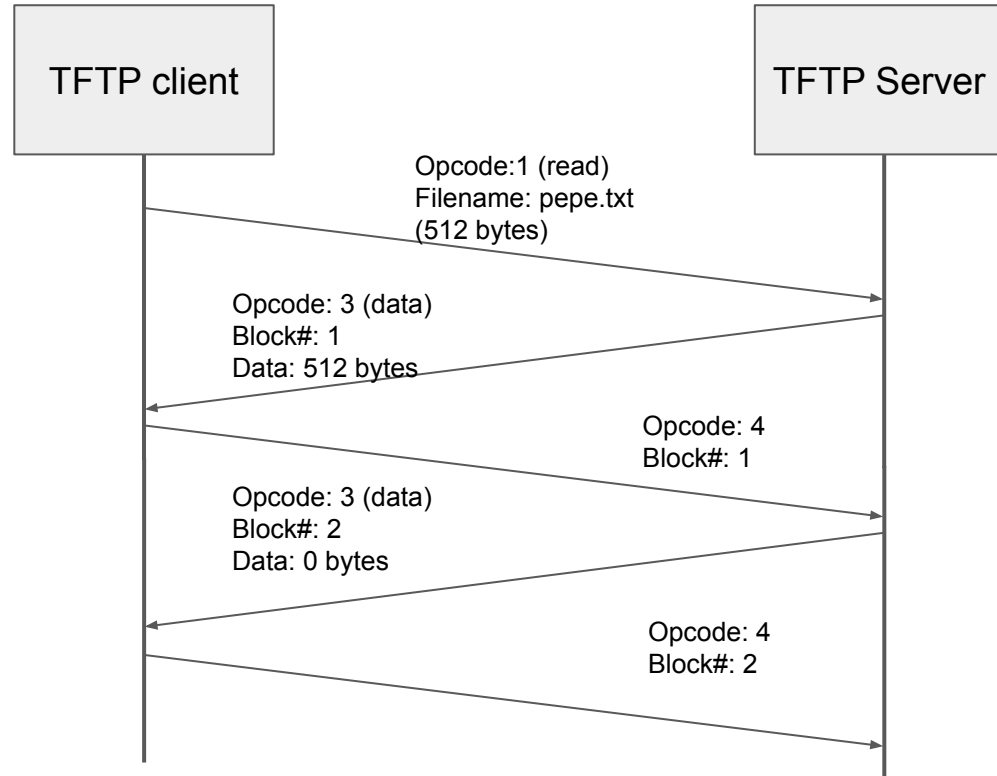


Mensajes | Longitud de Paquete (Ejemplo: TFTP)



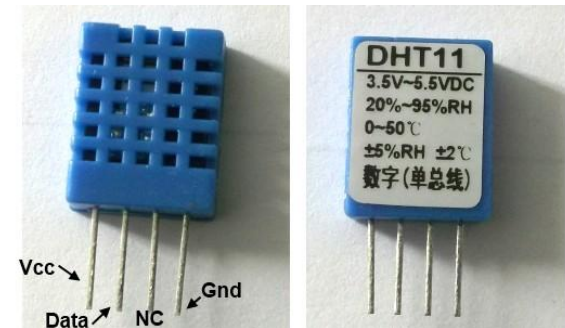
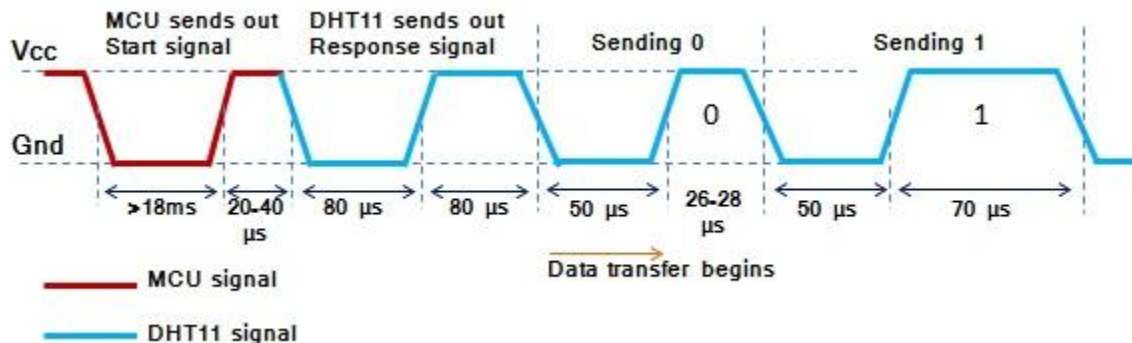


Mensajes | Longitud de Paquete (Ejemplo: TFTP)



Mensajes | Sincronización de señales

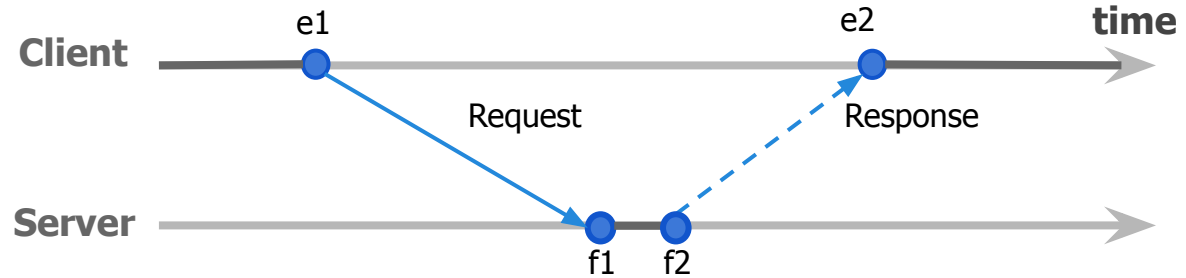
- En ciertos protocolos es necesario saber dónde empieza una señal para entender la secuencia de paquetes.
- **Ejemplo bajo nivel: DHT11 (sensor de humedad y temperatura)**
 - DHT11 envía data sobre pin de comunicación
 - Comienzo con una señal de sincronismo
 - Data (40-bit) = Integer Byte of RH + Decimal Byte of RH + Integer Byte of Temp. + Decimal Byte of Temp. + Checksum Byte



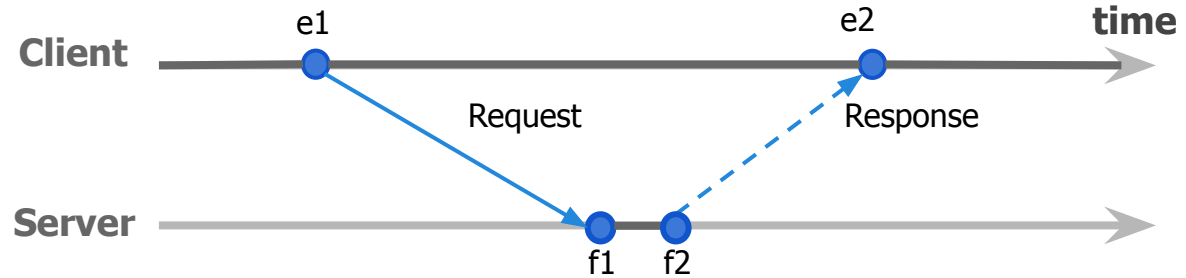


Mensajes | Sincrónicos vs Asincrónicos

Sincrónico



Asincrónico



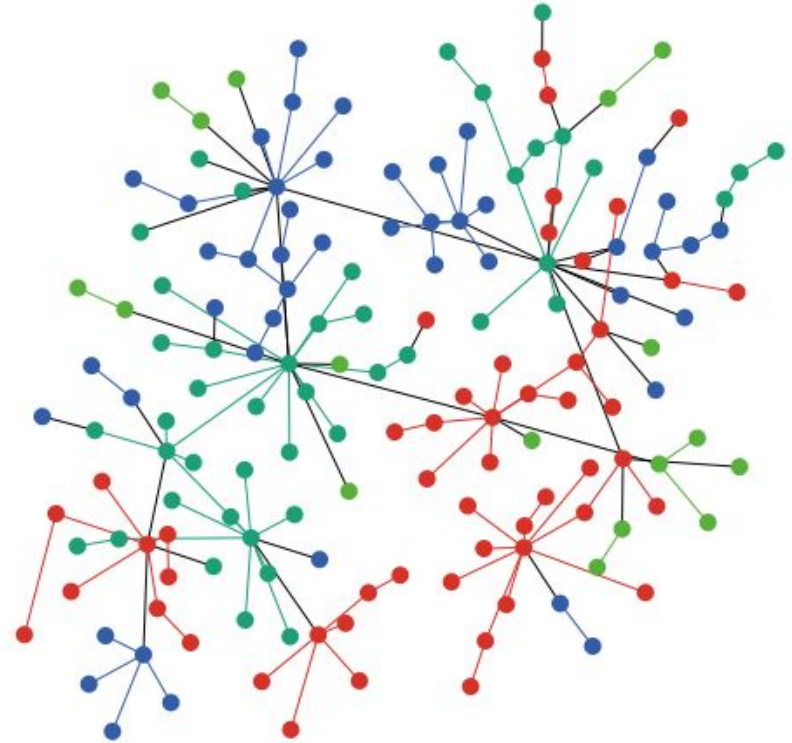
Agenda



- ☐ Mensajes
- ☒ **Grupos**
- ☐ Middlewares
- ☐ MOMs



- Permiten ver a una colección de procesos como una abstracción
- Mensaje es enviado a todas o algunas entidades que componen el grupo
- Grupos son dinámicos
 - Deben poder crearse y destruirse en todo momento
- Procesos se deben poder suscribir y cancelar la suscripción a grupos (deben existir primitivas para que esto suceda)



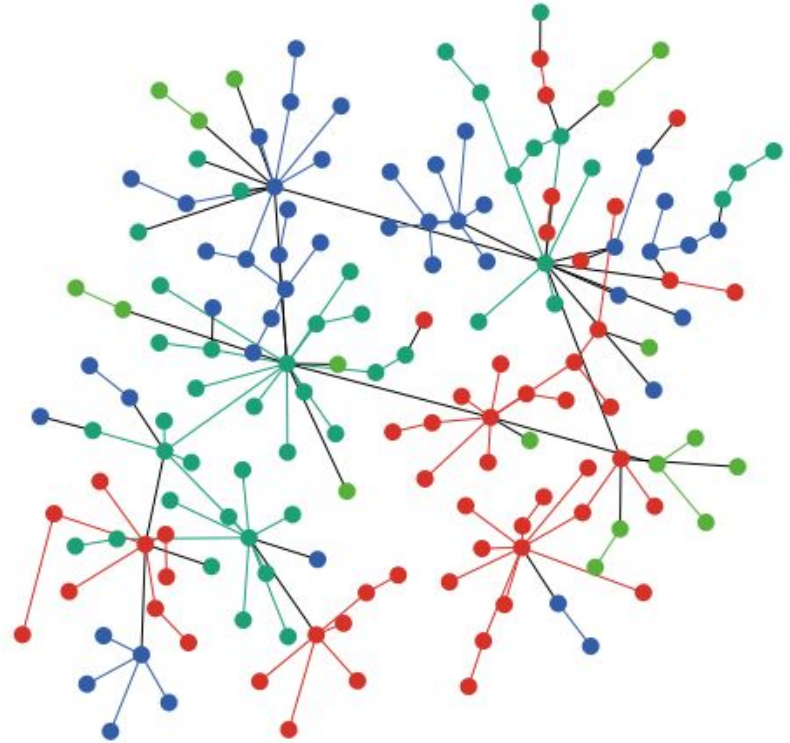


Uno a uno

- *Unicast*: Comunicación punto a punto
- *Anycast*: Uno sólo en el grupo recibe el mensaje
 - E.g.: Envío al nodo más cercano (ECMP). Que implica cercanía?

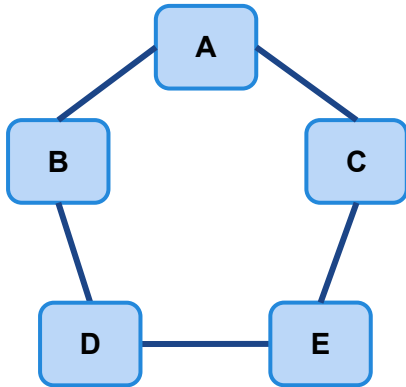
Uno a Muchos

- *Multicast*: Sólo aquellos que se encuentran en el grupo reciben el mensaje
- *Broadcast*: Todos reciben el mensaje

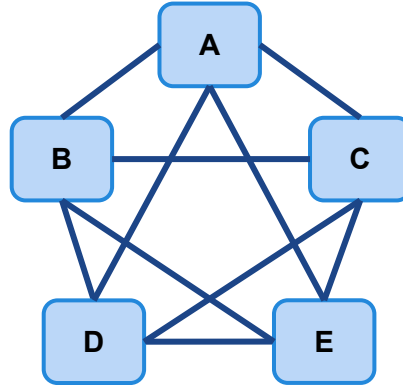




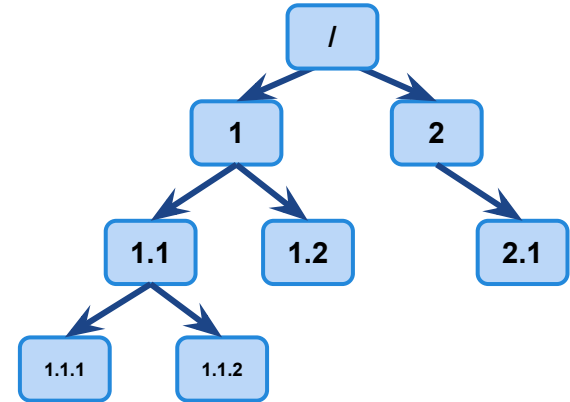
Anillos



Punto a Punto

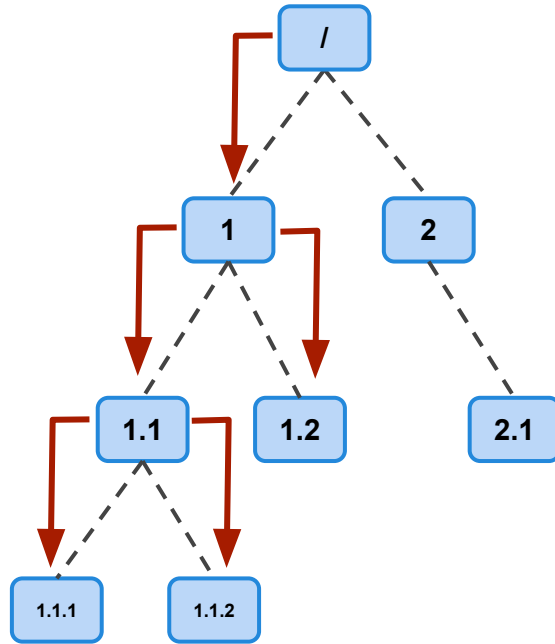


Grupos Jerárquicos

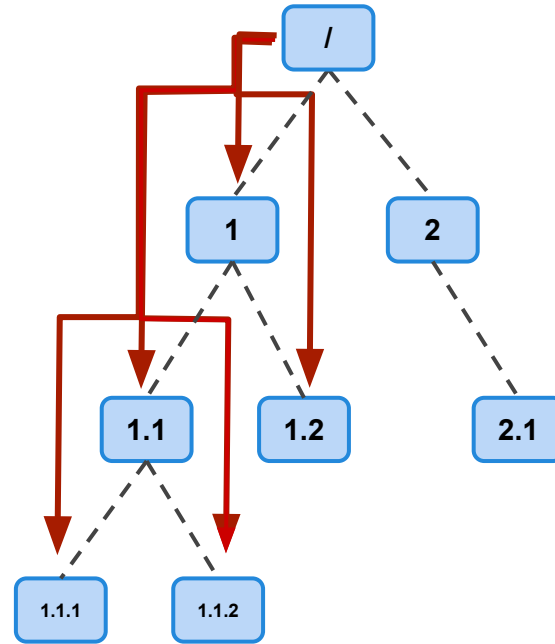




Difusión Descentralizada



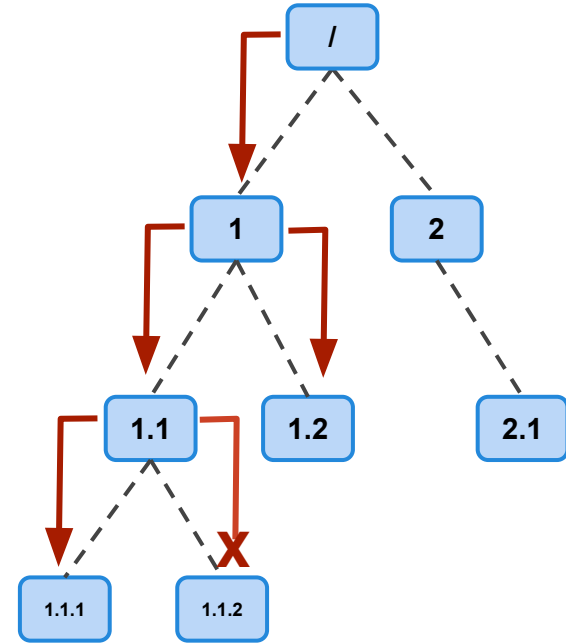
Difusión Centralizada





Grupos de Comunicación | Atomicidad de mensajes

- Los mensajes deben entregarse a todos o a ninguno de los miembros
- Necesidad de realizar ACK de mensajes
- Necesidad de demorar el *delivery* de los paquetes recibidos
- Reintentos frente a:
 - Caída de receptores
 - Caída del coordinador
 - No recepción de mensajes
 - No recepción de ACKs



Agenda



- ☐ Mensajes
- ☐ Grupos
- ☒ **Middleware**s
- ☐ MOMs



“[...] software de conectividad que ofrece un conjunto de servicios que hacen posible el funcionamiento de aplicaciones distribuidas sobre plataformas heterogéneas [...]”

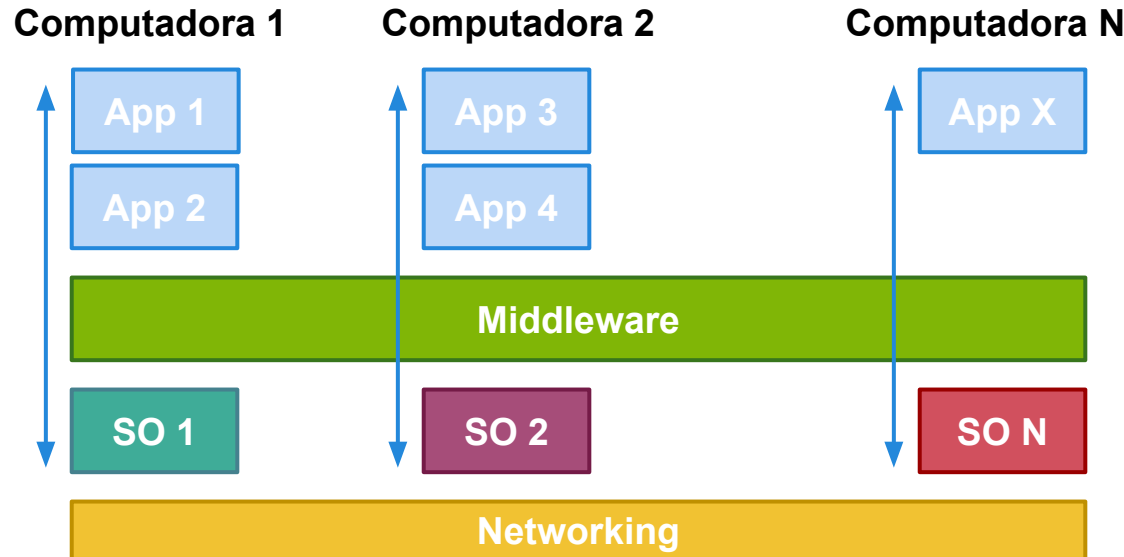
“[...] módulo intermedio que actúa como conductor entre sistemas permitiendo a cualquier usuario de sistemas de información comunicarse con varias fuentes de información que se encuentran conectadas por una red [...]”

“[...] capa de software que se encuentra o sitúa entre el sistema operativo y las aplicaciones del sistema [...]”

“[...] software que permite conectar componentes, softwares o aplicaciones. El mismo consiste en un conjunto de servicios que permiten que múltiples procesos corriendo en una o varias máquinas interactúen de un lado a otro de la red.”



Capa de software entre el sistema operativo y la capa de aplicación/usuario, para proveer una vista única del sistema.





Transparencia

- Se oculta la distribución y el sistema responde como si fuera una única computadora
- Respecto de: Acceso, Ubicación, Migración, Replicación, Concurrencia, Fallos, Persistencia

Tolerancia a Fallos

- Sistemas confiables, que se ejecuten y comporten de manera predecible incluso frente a la aparición de fallos.
- Abarcando: *Availability, Reliability, Safety, Maintainability*



Acceso a recursos compartidos

- Eficiente, transparente y controlado

Sistemas distribuidos abiertos (Interfaces)

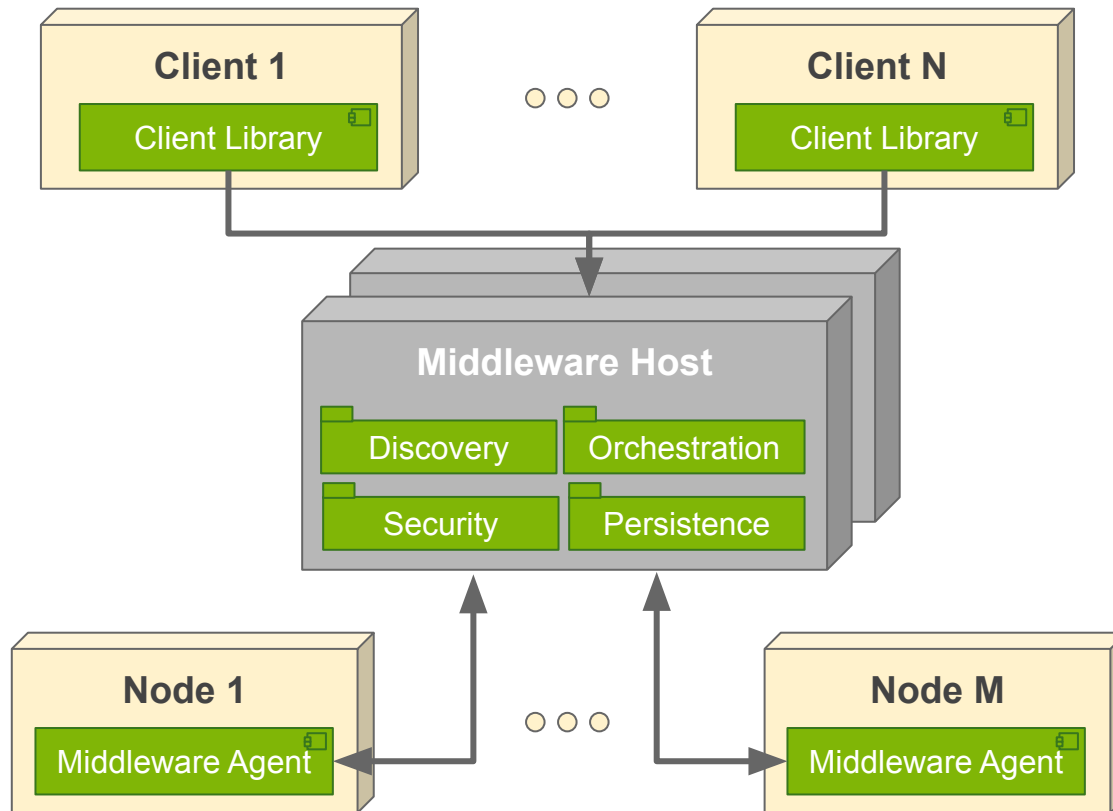
- Estándares claros sobre sintaxis y semántica de los servicios ofrecidos
- Interoperabilidad y portabilidad

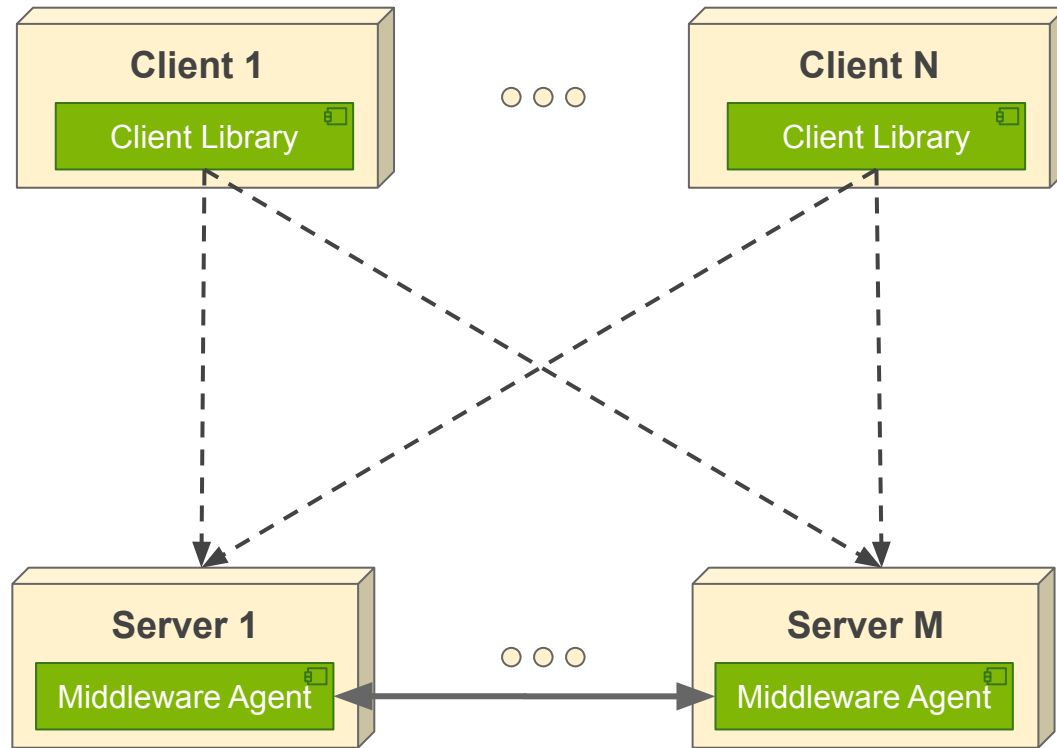
Comunicación de grupos

- Permite *broadcasting* y *multicasting*
- Facilita localización de elementos y coordinación de tareas



Middleware | Vista Física (centralizado)







- *Transactional Procedure*
- *Procedure Oriented*
- *Object Oriented*
- *Message Oriented*
- *Reflective Middlewares* (de configuración dinámica)

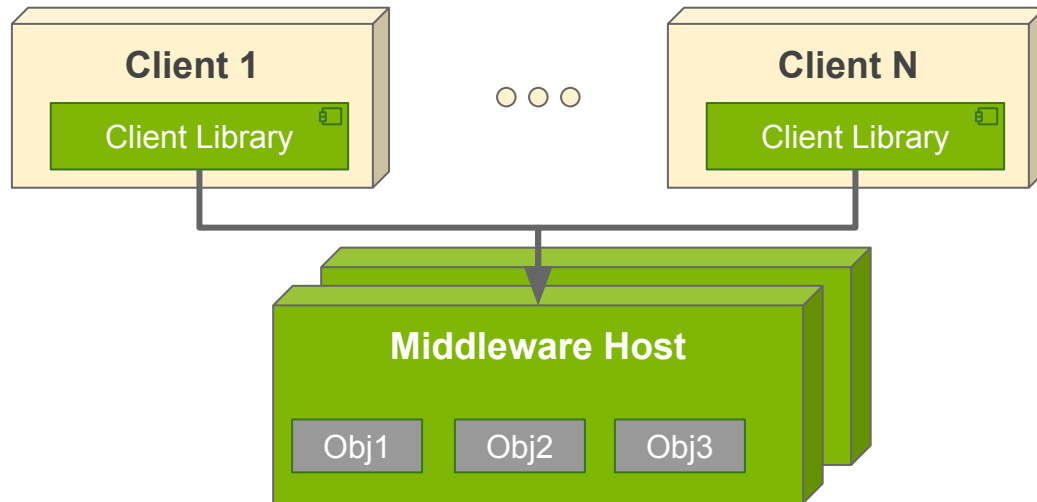


- Permiten garantizar transaccionalidad de operaciones respecto de datos
- Conectan muchas fuentes de datos y permiten un acceso transparente al grupo
- Poseen políticas de reintentos y retención de datos frente a caídas internas

Clasificación | *Object Oriented*



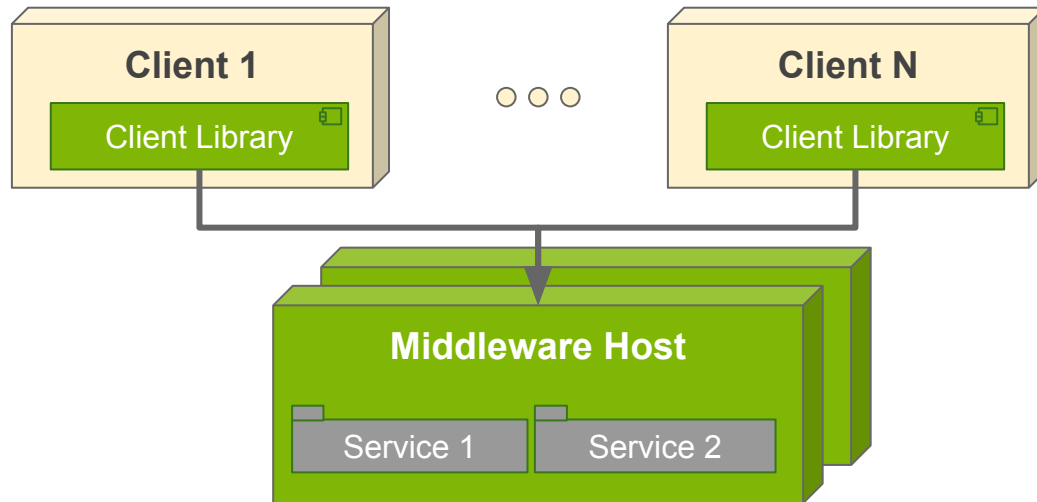
- Mensajes hacia objetos distribuidos
- Los objetos viven dentro del *middleware*
- Esquema de 'marshalling' para transmitir la información





Clasificación | *Procedure Oriented*

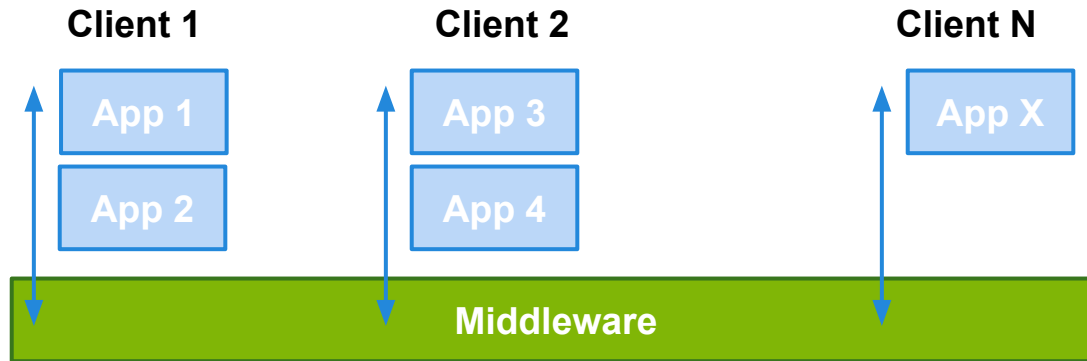
- El *middleware* trabaja como un servidor de funciones que se pueden invocar.
- Los servicios se pueden explorar y ejecutar pero no presentan estado para futuras invocaciones.

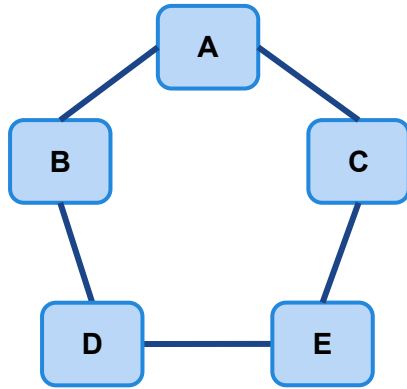




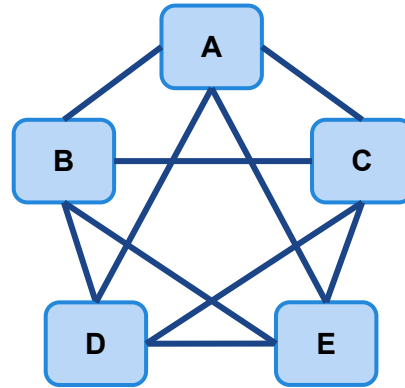
Clasificación | *Message Oriented*

- Funciona como un sistema de mensajería entre aquellas aplicaciones que utilizan el *middleware*
- Pueden enviarse mensajes bajo cierto 'tópico' para que aquellos interesados lo reciban (modo *Information Bus*)
- Pueden enviarse mensajes con un destinatario definido (modo *Queue*)

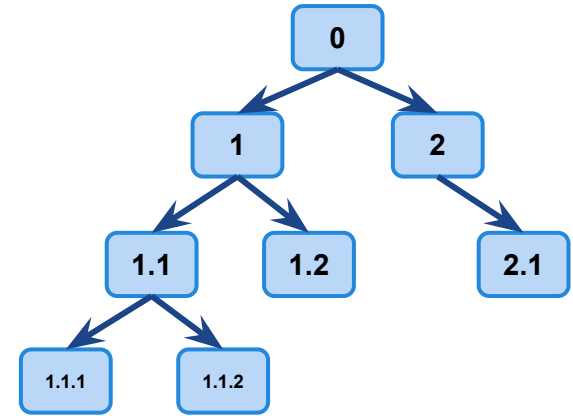




Anillos



Punto a Punto



Grupos Jerárquicos

Agenda

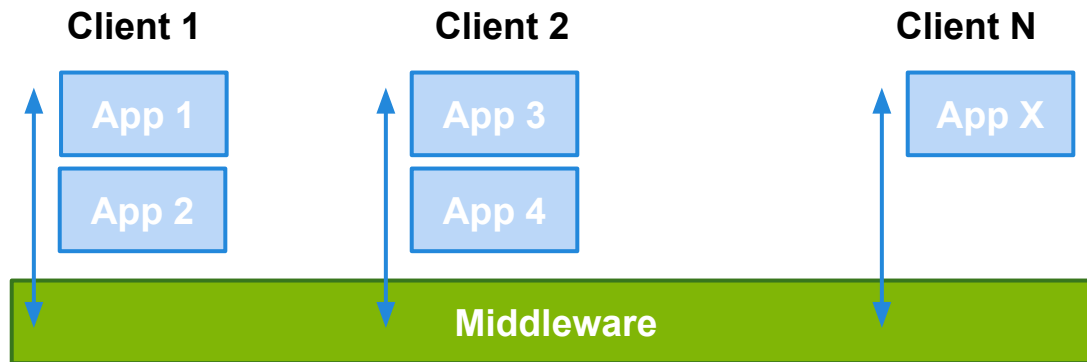


- ☐ Mensajes
- ☐ Grupos
- ☐ Middlewares
- ☒ **MOMs**



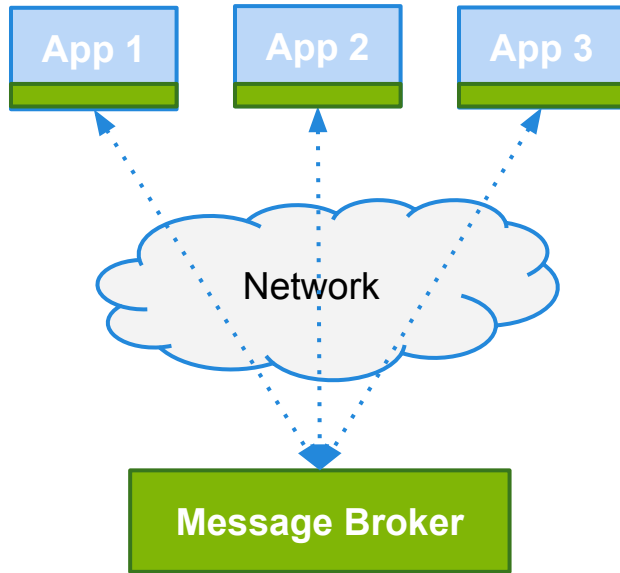
MOM | Introducción

- Implementan la comunicación de grupo de forma transparente a las aplicaciones que la requieren.
- Basan su funcionamiento en el simple concepto de comunicar mensajes entre aplicaciones.
- Resuelve problemas de transparencia respecto de ubicación, fallos, performance y escalabilidad.

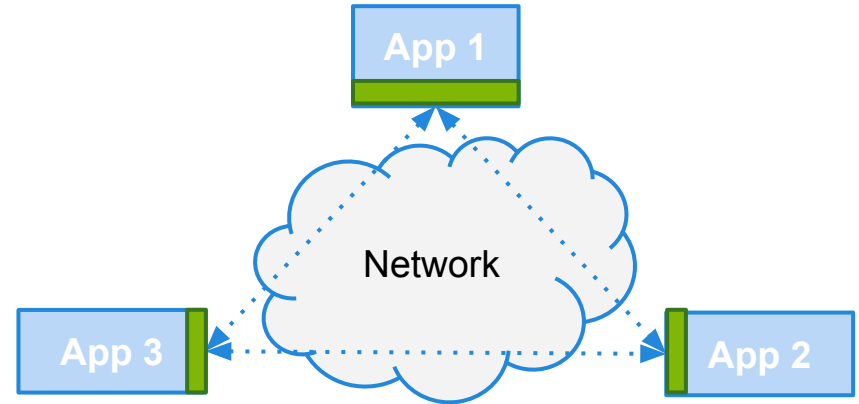




Centralizado



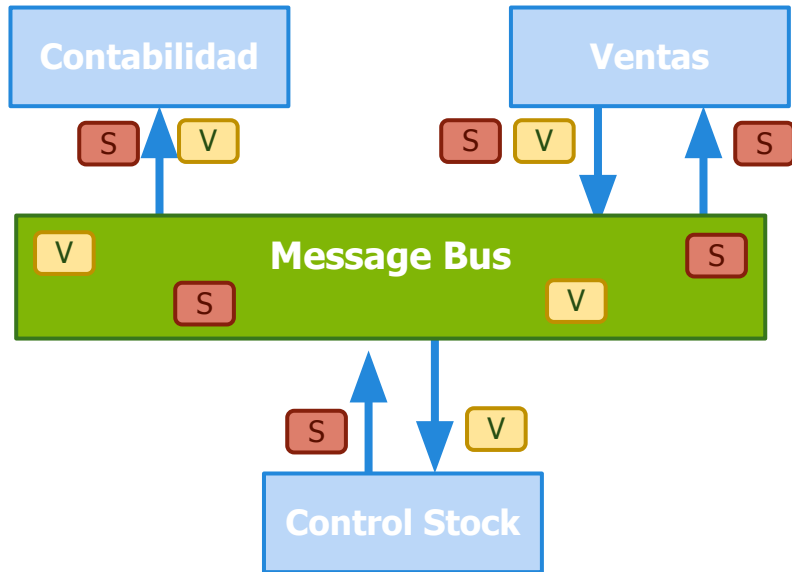
Distribuido



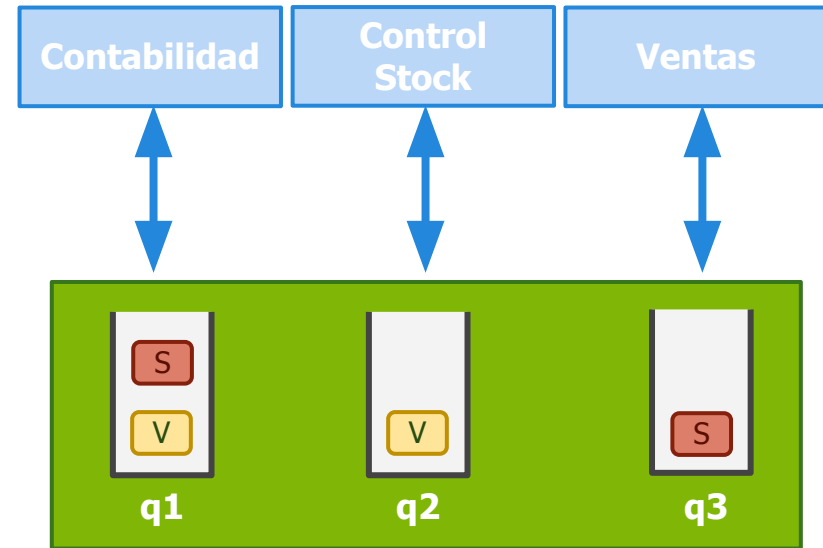


MOM | Bus de Información vs Colas de Mensajes

Bus



Colas





Pros

- Se modela como una conexión punto a punto
- Permite obtener respuestas instantáneas a pedidos concretos

Contras

- No permite implementar transparencia frente a errores

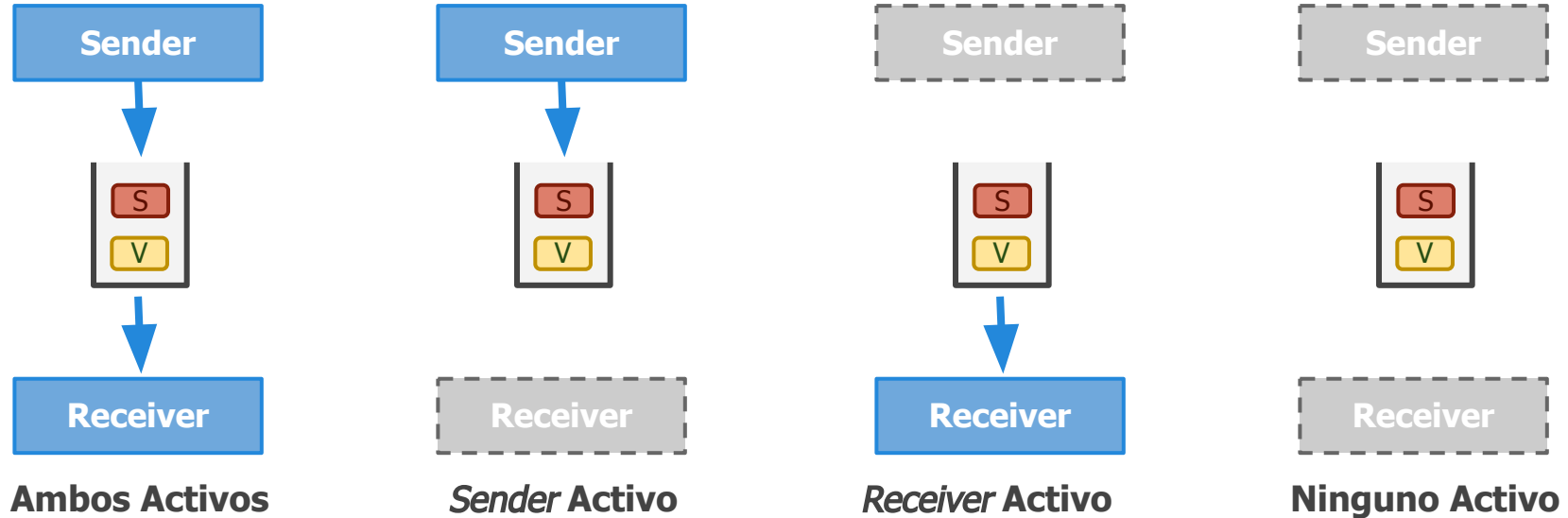




MOM | Modelo Asíncrono del MOM

Pros

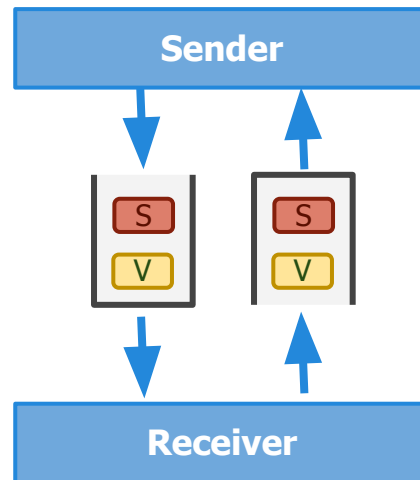
- Se modela naturalmente con colas
- La arquitectura soporta períodos de discontinuidad del transporte





Contras

- Es complejo recibir respuesta a pedidos realizados (mínimamente es necesario contar con colas para el retorno de info)



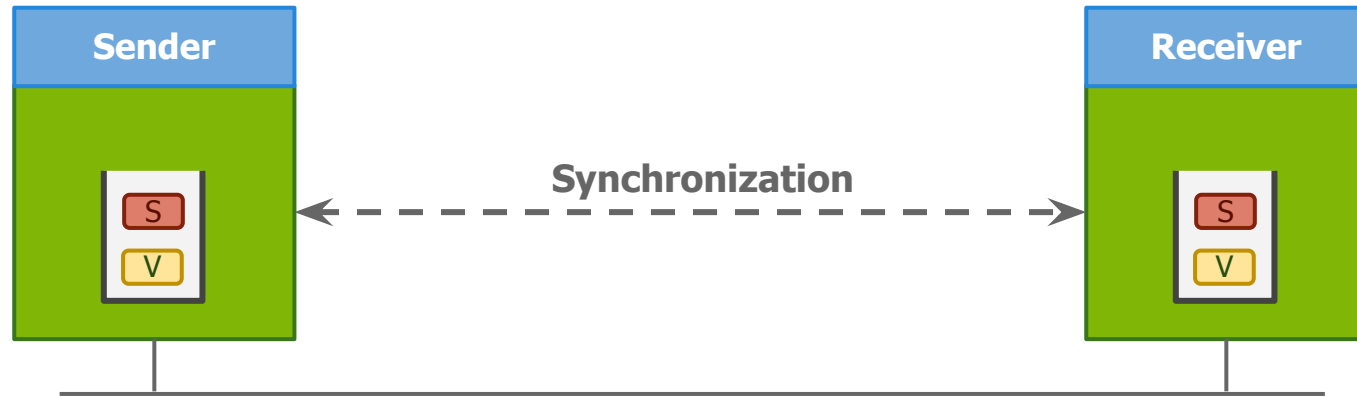


- **put:** publicación de un cierto mensaje
- **get:** esperar hasta que un mensaje sea detectado. Luego, eliminarlo de la cola y retornarlo
- **poll:** revisar mensajes pendientes, sin bloquear
- **notify:** asociar un *callback* utilizado por el MOM para ser ejecutado frente a publicación de ciertos mensajes



MOM | Colas de Mensajes

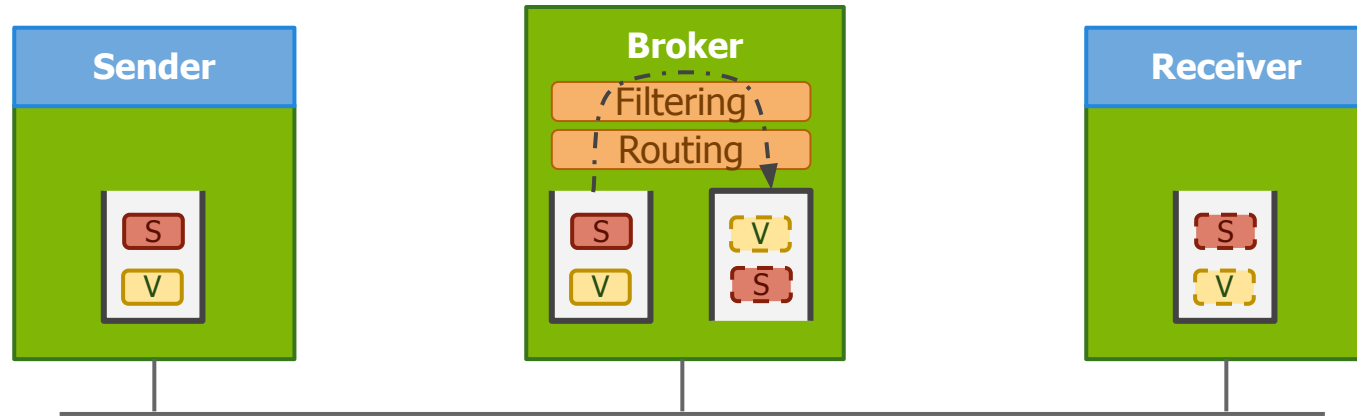
- Pueden existir varias definidas dentro del MOM
- Tienen nombre y longitud definidas
- Los clientes suelen contar con colas privadas intermedias
- Garantía al Emisor de que el mensaje será insertado





MOM | Brokers

- Proveen transparencia de localización tanto al Emisor como al Receptor.
- Soportan lógica en el *middleware* para filtrar y modificar mensajes.
- Brindan un punto de control y monitoreo





- P. Verissimo, L. Rodriguez: Distributed Systems for Systems Architects, Kluwer Academic Publishers, 2001.
 - Capítulo 2.1: Naming and addressing
 - Capítulo 2.4: Group Communication
- P. Bernstein. Middleware - An Architecture for Distributed System Services, Digital Equipment Corporation. Cambridge Research Lab. 1993.
- T. Bishop and R. Karne. "A Survey of Middleware." Computers and Their Applications. 2003.
- M. Van Steen, A. Tanenbaum: Distributed Systems. 3rd Edition. Pearson Education, 2017.
 - Capítulo 4.3: Message-Oriented Communication.